



# AUDIT REPORT

---

May 2026

For

**YELLOW**ART

# Table of Content

|                                                                                                              |    |
|--------------------------------------------------------------------------------------------------------------|----|
| Executive Summary                                                                                            | 03 |
| Number of Security Issues per Severity                                                                       | 04 |
| Summary of Issues                                                                                            | 05 |
| Checked Vulnerabilities                                                                                      | 06 |
| Techniques and Methods                                                                                       | 08 |
| Types of Severity                                                                                            | 10 |
| Types of Issues                                                                                              | 11 |
| Severity Matrix                                                                                              | 12 |
|  <b>Low Severity Issues</b> | 13 |
| 1. Token URI is set after _safeMint receiver callback                                                        | 13 |
| 2. ERC20 transferFrom is incompatible with non-standard tokens                                               | 14 |
| Accepted Design Assumptions                                                                                  | 15 |
| Functional Tests                                                                                             | 16 |
| Automated Tests                                                                                              | 16 |
| Threat Model                                                                                                 | 17 |
| Closing Summary & Disclaimer                                                                                 | 23 |



# Executive Summary

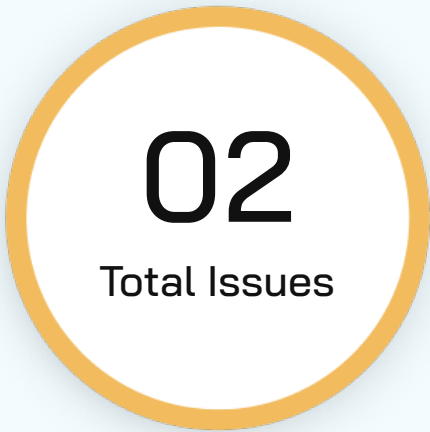
|                              |                                                                                                                                                                                  |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Project Name</b>          | Polarbit                                                                                                                                                                         |
| <b>Project Type</b>          | NFT                                                                                                                                                                              |
| <b>Project URL</b>           | <a href="https://yellowart.top/">https://yellowart.top/</a>                                                                                                                      |
| <b>Overview</b>              | ClearoNFT mints ERC721 NFTs through a trusted Transak NFT Checkout proxy. The proxy holds USDC, validates checkout data, calls mint, and funds the treasury payment.             |
| <b>Audit Scope</b>           | The scope of this audit was to analyze the Polarbit Smart Contracts for quality, security, and correctness.                                                                      |
| <b>Source Code link</b>      | <a href="https://repo.sourcify.dev/1/0x3477DcD1D05bD2C314dfCaa92892cF1184aaEc36">https://repo.sourcify.dev/1/0x3477DcD1D05bD2C314dfCaa92892cF1184aaEc36</a>                      |
| <b>Contracts in Scope</b>    | contracts/ClearoNFT.sol                                                                                                                                                          |
| <b>Branch</b>                | Deployed on Mainnet                                                                                                                                                              |
| <b>Commit Hash</b>           | <a href="https://etherscan.io/address/0xa4596ce4a7d77941f91501e24519dbaca7631f32685b85dbf693289744437f4e">0xa4596ce4a7d77941f91501e24519dbaca7631f32685b85dbf693289744437f4e</a> |
| <b>Language</b>              | Solidity                                                                                                                                                                         |
| <b>Blockchain</b>            | Ethereum                                                                                                                                                                         |
| <b>Method</b>                | Manual Analysis, Functional Testing, Automated Testing                                                                                                                           |
| <b>Review 1</b>              | 30th April, 2026                                                                                                                                                                 |
| <b>Updated Code Received</b> | 19th May 2026                                                                                                                                                                    |
| <b>Review 2</b>              | 19th May 2026                                                                                                                                                                    |
| <b>Fixed In</b>              | <a href="https://etherscan.io/address/0xE339835F6098b971b00238f4F7c5c9512cbDE2b8#code">https://etherscan.io/address/0xE339835F6098b971b00238f4F7c5c9512cbDE2b8#code</a>          |

Verify the Authenticity of Report on QuillAudits Leaderboard:

<https://www.quillaudits.com/leaderboard>



# Number of Issues per Severity



|               |            |
|---------------|------------|
| Critical      | 0 (0.0%)   |
| High          | 0 (0.0%)   |
| Medium        | 0 (0.0%)   |
| Low           | 2 (100.0%) |
| Informational | 0 (0.0%)   |

|        |                    | Severity |      |        |     |               |
|--------|--------------------|----------|------|--------|-----|---------------|
|        |                    | Critical | High | Medium | Low | Informational |
| Issues | Open               | 0        | 0    | 0      | 0   | 0             |
|        | Acknowledged       | 0        | 0    | 0      | 0   | 0             |
|        | Partially Resolved | 0        | 0    | 0      | 0   | 0             |
|        | Resolved           | 0        | 0    | 0      | 2   | 0             |



# Summary of Issues

| Issue No. | Issue Title                                                 | Severity | Status   |
|-----------|-------------------------------------------------------------|----------|----------|
| 1         | Token URI is set after _safeMint receiver callback          | Low      | Resolved |
| 2         | ERC20 transferFrom is incompatible with non-standard tokens | Low      | Resolved |



# Checked Vulnerabilities

✓ Access Management

✓ Arbitrary write to storage

✓ Centralization of control

✓ Ether theft

✓ Improper or missing events

✓ Logical issues and flaws

✓ Arithmetic Computations  
Correctness

✓ Race conditions/front running

✓ SWC Registry

✓ Re-entrancy

✓ Timestamp Dependence

✓ Gas Limit and Loops

✓ Exception Disorder

✓ Gasless Send

✓ Use of tx.origin

✓ Malicious libraries

✓ Compiler version not fixed

✓ Address hardcoded

✓ Divide before multiply

✓ Integer overflow/underflow

✓ ERC's conformance

✓ Dangerous strict equalities

✓ Tautology or contradiction

✓ Return values of low-level calls



✓ Missing Zero Address Validation

✓ Private modifier

✓ Revert/require functions

✓ Multiple Sends

✓ Using suicide

✓ Using delegatecall

✓ Upgradeable safety

✓ Using throw

✓ Using inline assembly

✓ Style guide violation

✓ Unsafe type inference

✓ Implicit visibility level



# Techniques and Methods

**Throughout the audit of smart contracts, care was taken to ensure:**

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

**The following techniques, methods, and tools were used to review all the smart contracts:**

## Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

## Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.





### Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

### Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

### Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.



# Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

## ■ **Critical: Immediate and Catastrophic Impact**

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

## ■ **High (H): Significant Risk of Major Loss or Compromise**

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

## ■ **Medium (M): Potential for Moderate Harm Under Specific Circumstances**

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

## ■ **Low (L): Minor Imperfections with Limited Repercussions**

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

## ■ **Informational (I): Opportunities for Improvement, Not Immediate Risks**

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.



# Types of Issues

## Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

## Resolved

These are the issues identified in the initial audit and have been successfully fixed.

## Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

## Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

# Severity Matrix

|            |        | Impact   |        |        |
|------------|--------|----------|--------|--------|
|            |        | High     | Medium | Low    |
| Likelihood | High   | Critical | High   | Medium |
|            | Medium | High     | Medium | Low    |
|            | Low    | Medium   | Low    | Low    |

## Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

## Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.



# Low Severity Issues

## Token URI is set after `_safeMint` receiver callback

**Resolved**

### Path

contracts/ClearoNFT.sol

### Function Name

`mint()`

### Description

The mint function calls `_safeMint(buyer, tokenId)` before `_setTokenURI(tokenId, tokenURI_)`.

```
uint256 tokenId = nextTokenId++;  
_safeMint(buyer, tokenId);  
_setTokenURI(tokenId, tokenURI_);
```

OpenZeppelin's `_safeMint` first mints the token and then calls `onERC721Received` if buyer is a contract. This means a contract recipient can observe the token during the receiver callback before its metadata URI has been stored.

### Impact

Contract recipients, marketplace escrow contracts, or smart wallets that read `tokenURI(tokenId)` during `onERC721Received` may observe an empty URI. This can cause integration failures or incorrect metadata-dependent behavior.

### Likelihood

Low

### Recommendation

Make the `_setTokenURI` precede the `_safeMint` call.



## ERC20 transferFrom is incompatible with non-standard tokens

**Resolved**

### Path

contracts/ClearoNFT.sol

### Function Name

`mint()`

### Description

The contract uses raw `IERC20.transferFrom` for USDC payment settlement.

```
require(usdc.transferFrom(msg.sender, treasury, price), "USDC transfer failed");
```

This works for standard ERC20 tokens that return `bool`, including canonical USDC. However, the token address is constructor-configurable, so the contract may be deployed with a non-standard ERC20, bridged USDC variant, or token that does not return a boolean value.

### Impact

If deployment on other chains use a USDC contract which does not behave like mainnet USDC, minting could fail unexpectedly.

### Likelihood

Low

### Recommendation

Use OpenZeppelin's `SafeERC20.safeTransferFrom` to support standard and commonly encountered non-standard ERC20 behavior.



# Accepted Design Assumptions

| Design Choice                     | Client's Comments                                                                                                                              |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Variable NFT price                | Price is variable and validated upon proxy calling.                                                                                            |
| Token ID 0                        | Token ID #0 can be read from the marketplace we are listing on.                                                                                |
| Per-token URI                     | tokenURI is the metadata uri associated with each minted token. It is used in the marketplace to define name, img, and attributes.             |
| Trusted proxy                     | It is passed in parameters by the callee, eg. passed by us to Transak NFT Checkout which then passes it as a parameter to contract.            |
| USDC custody and Treasury funding | Proxy is used as we are using Transak NFT Checkout. We ensure the callee is the official Transak-owned address.<br><br>Performed by the proxy. |



# Functional Tests

Some of the tests performed are mentioned below:

- ✓ TransakProxy should be able to mint tokens.

# Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.





# Threat Model

## 1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

### 1.1 External Dependencies Table

| Dependency                 | Type                                       | How It Is Used                                                                                                                        | Trust Assumption                                                                                                          | Associated Risks                                                                                                                                             |
|----------------------------|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Transak NFT Checkout Proxy | External trusted minter / payment executor | Sole caller authorized by onlyTransak; passes buyer, treasury, price, and tokenURI_ to mint; holds or controls USDC used for payment. | The configured transakProxy is the official Transak-owned address and validates checkout data before calling mint.        | Wrong proxy address can permanently disable minting or authorize the wrong caller. Proxy misconfiguration can pass incorrect buyer, treasury, price, or URI. |
| USDC Token                 | External ERC20 token                       | ClearoNFT calls usdc.transferFrom(msg.sender, treasury, price) to move payment from the proxy to the treasury.                        | The configured token is the intended USDC contract for the target chain and behaves like a standard ERC20 returning bool. | Wrong token address, blacklist, pause, insufficient allowance, insufficient balance, or non-standard token behavior can make minting revert.                 |



| Dependency                       | Type                         | How It Is Used                                                                               | Trust Assumption                                                                                      | Associated Risks                                                                                                                                                         |
|----------------------------------|------------------------------|----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OpenZeppelin<br>ERC721URIStorage | Third-party contract library | Provides ERC721 ownership, transfer, metadata, ERC165, _safeMint, and _setTokenURI behavior. | The inherited OpenZeppelin implementation behaves as documented and is used without unsafe overrides. | _safeMint invokes contract recipient callbacks before ClearNFT writes token metadata. Library version and compiler version should be pinned for reproducible deployment. |
| Buyer<br>(Recipient)             | External callback target     | If buyer is a contract, _safeMint invokes onERC721Received.                                  | Contract recipients either accept the NFT or intentionally revert.                                    | Recipient can revert and block its own mint. Recipient can read state during callback before tokenURI_ is stored.                                                        |



## 2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

### 2.1 Function Entry / Exit Table

Each function gets one table.

| Contract Name | Function                                          | Category                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|---------------|---------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ClearoNFT     | constructor(address _transakProxy, address _usdc) | <p><b>What this function can do</b><br/>Initializes the ERC721 collection name and symbol. Stores the immutable Transak proxy address and immutable USDC token address.</p> <p><b>What this function cannot/should not do</b><br/>Cannot be called again after deployment. Should not accept address(0) for either _transakProxy or _usdc because both addresses are immutable.</p> <p><b>Main invariant(s)</b><br/>transakProxy and usdc are fixed for the lifetime of the contract. The collection identity is Clearo NFT / CNFT after deployment.</p> <p><b>Access level</b><br/>Deployment-time only.</p> |



| Contract Name | Function                                                               | Category                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|---------------|------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ClearoNFT     | mint(address buyer, address treasury, uint256 price, string tokenURI_) | <p><b>What this function can do</b></p> <p>Transfers price USDC from the Transak proxy to treasury, increments nextTokenId, mints one ERC721 token to buyer, and stores the token-specific metadata URI.</p> <p><b>What this function cannot/should not do</b></p> <p>Cannot be called by any address other than transakProxy. Should not independently validate checkout price, buyer eligibility, or URI correctness because those checks are delegated to the trusted Transak flow.</p> <p><b>Main invariant(s)</b></p> <p>Only the configured proxy can mint. Each successful mint increments nextTokenId once and creates exactly one NFT. Payment transfer occurs before NFT minting. Token IDs start at 0.</p> <p><b>Access level</b></p> <p>external; restricted by onlyTransak.</p> |

### 3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

#### 3.1 Asset-Holding Contracts

List only contracts that custody value.

| Contract                                            | Asset Type                                            | Custodied Assets                                                                                                                                                                                                                             |
|-----------------------------------------------------|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Transak NFT Checkout Proxy (external, out of scope) | USDC                                                  | Holds or controls USDC before calling ClearoNFT.mint; payment is pulled from this proxy into the treasury.                                                                                                                                   |
| ClearoNFT                                           | ERC721 minting authority, no intended payment custody | Does not intentionally custody USDC or native ETH in the mint flow. It creates NFTs and transfers USDC directly from proxy to treasury. Any accidental ERC20/native transfer to this contract has no recovery function in the reviewed code. |



### 3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

| Contract     | Function                                                               | Asset In                                                                                         | Asset Out                                                                             | Caller                      | Risk Notes                                                                                                                                                                                           |
|--------------|------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ClearoNFT    | mint(address buyer, address treasury, uint256 price, string tokenURI_) | No USDC enters ClearoNFT ; price USDC is pulled from the proxy. One new ERC721 token is created. | price USDC moves from Transak proxy to treasury; one ERC721 token is minted to buyer. | transakProxyonly.           | Raw ERC20 transferFrom depends on configured token behavior. Buyer contract receives a callback before metadata is stored. Incorrect immutable deployment addresses can permanently break this flow. |
| VotingEscrow | depositFor()                                                           | ELSA from msg.sender (safeTransferFrom), credited to addr's lock                                 | –                                                                                     | Any non-blacklisted address | Blocked for eligible recipients after eligibilityEnd; both msg.sender and addr checked against blacklist; token flows from depositor not lock owner                                                  |



# Closing Summary

In this report, we have considered the security of Polarbit. We performed our audit according to the procedure described above.

2 low severity issues were found and the Polarbit Team resolved them all.

# Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.



# About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers.

With eight years of expertise, we've secured over 1500 projects globally, averting over \$3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

**8+**

Years of Expertise

**1M+**

Lines of Code Audited

**50+**

Chains Supported

**1500+**

Projects Secured

Follow Our Journey





# AUDIT REPORT

---

May 2026

For

**YELLOW**ART



Canada, India, Singapore, UAE, UK

[www.quillaudits.com](http://www.quillaudits.com)

[audits@quillaudits.com](mailto:audits@quillaudits.com)